

Events Platform API

Beginner Guide: How to Run & How It Works

A simple walkthrough so you can start the server, try the APIs, understand auth/OTP/events/enrollment, and finish your assignment work.

[OK] Project folder: ~/events-platform

[OK] Stack: Django + DRF + PostgreSQL + SimpleJWT

[OK] Docs UI: <http://127.0.0.1:8000/api/docs/>

1. What is this project?

This is a backend API for an Events Platform. There is no fancy website homepage - clients talk to JSON endpoints. Facilitators create events; Seekers search, enroll, cancel, and re-enroll. Users sign up with email, verify a 6-digit OTP, then get JWT tokens.

Two roles

- Facilitator - create / update / delete own events; see enrollment_count and available_seats
- Seeker - search events; enroll / cancel; list upcoming or past enrollments

Important folders

- accounts/ - signup, login, OTP, Profile (role linked 1-to-1 with Django User)
- events/ - Event + Enrollment models, search, enroll with DB locking
- config/ - Django settings, URLs, standardized error format
- chaos/ - scripts that deliberately break things (for DEBUGGING.md evidence)
- docs/ - API proof instructions (screenshots)

2. Do I need to change .env?

For the default local setup with Docker Postgres on port 5433: NO. Your .env is already correct.

Current expected values

```
DJANGO_DEBUG=True
DJANGO_ALLOWED_HOSTS=localhost,127.0.0.1,testserver
POSTGRES_HOST=127.0.0.1
POSTGRES_PORT=5433
POSTGRES_USER=events
POSTGRES_PASSWORD=events
POSTGRES_DB=events_platform
EMAIL_BACKEND=django.core.mail.backends.console.EmailBackend
```

Note: OTP emails are printed in the terminal (console backend). You do not need Gmail/SMTP for this assignment.

Change .env only if you use a different Postgres host/port/password, or when deploying to production (strong SECRET_KEY, DEBUG=False).

3. How to run (step by step)

Step A - Start PostgreSQL

```
docker start events-pg
# If the container does not exist yet:
docker run -d --name events-pg \
  -e POSTGRES_USER=events -e POSTGRES_PASSWORD=events \
  -e POSTGRES_DB=events_platform -p 5433:5432 postgres:16-alpine
```

Step B - Activate venv and start server

```
cd ~/events-platform
source .venv/bin/activate
python manage.py migrate
```

```
python manage.py seed_demo  
python manage.py runserver
```

Step C - Open these URLs

- <http://127.0.0.1:8000/> -> redirects to Swagger docs
- <http://127.0.0.1:8000/api/docs/> -> interactive API explorer
- <http://127.0.0.1:8000/admin/> -> Django admin

Note: A 404 on <http://127.0.0.1:8000/> before the redirect fix was normal - this is an API project, not a website. Use [/api/docs/](http://127.0.0.1:8000/api/docs/).

Demo accounts (already verified by seed_demo)

```
Facilitator: facilitator@example.com / Passw0rd!  
Seeker:      seeker1@example.com      / Passw0rd!  
              seeker2@example.com ... seeker5@example.com
```

4. How everything works (big picture)

Request flow: Client (Swagger / Postman) -> Django URLs -> Views/Serializers -> Services (OTP / enrollment locking) -> PostgreSQL

Standard responses

- Pagination shape: { count, next, previous, results }
- Errors: { "detail": "message", "code": "error_code_string" }
- Auth header: Authorization: Bearer <access_token>

4.1 Authentication & OTP

Signup does NOT accept username in the body - only email, password, role.

- POST /api/auth/signup/ - creates unverified user + Profile; emails OTP
- OTP stored as SHA-256 hash only (never returned in API JSON)
- POST /api/auth/verify-otp/ - marks email verified
- POST /api/auth/resend-otp/ - revokes old OTP; 60s cooldown; 5 min TTL; max 3 fails
- POST /api/auth/login/ - verified email + password -> access + refresh JWTs
- POST /api/auth/token/refresh/ - new access token

Unverified users cannot log in (code: email_not_verified). Requesting OTP #2 immediately invalidates OTP #1.

4.2 Events & search

- Facilitator CRUD: /api/facilitator/events/
- List includes enrollment_count and available_seats (null capacity = unlimited)
- Search: GET /api/events/?q=&location=&language=&starts_after=&starts_before=
- Default order: starts_at ascending (upcoming first)

4.3 Enrollment (the hard parts)

Concurrency (Challenge A)

When many seekers enroll at once, the service locks the Event row with select_for_update() inside transaction.atomic(), then recounts ENROLLED seats. Active enrollments can never exceed capacity.

Cancel & re-enroll (Challenge B)

Cancel sets status=CANCELLED (row kept as audit). Re-enroll inserts a NEW ENROLLED row. A partial unique constraint allows only one ENROLLED row per (event, seeker), while many CANCELLED rows are allowed.

5. What you should do for your work

Checklist

- 1. Start Postgres + runserver (Section 3)
- 2. Open Swagger; login as facilitator and seeker
- 3. Create/list events; enroll; cancel; re-enroll
- 4. Trigger capacity_full error (capacity=1 event, two seekers)
- 5. Run automated tests (below)
- 6. Optional: run chaos scripts and keep logs for DEBUGGING.md
- 7. Optional: Postman screenshots (see docs/API_PROOF.md)

Try in Swagger

- 1) Call POST /api/auth/login/ with seeker1 credentials.
- 2) Copy the access token.
- 3) Click Authorize -> paste: Bearer <access>
- 4) Call GET /api/events/ and POST /api/events/{id}/enroll/

Run tests

```
python manage.py test accounts.tests events.tests -v 2
# Expect: Ran 17 tests ... OK
```

Chaos experiments (optional but great for understanding)

```
python chaos/chaos_a_concurrency.py
# Shows overbooking when locking is removed

python chaos/chaos_b_unique_together.py
# Shows IntegrityError on re-enroll with naive unique_together
```

Read DEBUGGING.md and DECISIONS.md - they explain the real failures and design choices.

6. Postman

- Import events_platform.postman_collection.json
- Set baseUrl = http://127.0.0.1:8000
- Login requests auto-save accessToken into collection variables
- Other requests use Authorization: Bearer {{accessToken}}

7. Key API map (short)

```
POST /api/auth/signup/
POST /api/auth/verify-otp/
POST /api/auth/resend-otp/
POST /api/auth/login/
POST /api/auth/token/refresh/
GET /api/auth/me/
GET/POST /api/facilitator/events/
GET/PATCH/DELETE /api/facilitator/events/{id}/
GET /api/events/ (search)
```

```
POST /api/events/{id}/enroll/
POST /api/events/{id}/cancel/
GET /api/enrollments/me/?scope=upcoming|past|all
```

8. Common problems

Page not found on /

Use /api/docs/. Root now redirects there if you pulled the latest urls.py.

Could not connect to database

Start Docker Postgres: docker start events-pg. Confirm PORT=5433 in .env.

Login says email_not_verified

New signups must verify OTP from the runserver console. Seeded demo users are already verified.

capacity_full

Expected when the event is full - not a crash. Error code proves validation works.

9. Docs to read (in this order)

- README.md - setup & route map
- This PDF - your day-to-day guide
- DECISIONS.md - why locking / partial unique / OTP policy
- DEBUGGING.md - real chaos failure traces
- PROMPT_LOG.md - AI interaction log + corrections
- docs/API_PROOF.md - screenshot instructions

[OK] You are ready: start runserver -> open /api/docs/ -> login -> explore -> run tests.